DIÁRIO DE BORDO

EP - SISTEMAS OPERACIONAIS

Rômulo Maurício Fonseca Júnior - 15553801

Entrada 1: Início da Implementação e o Primeiro Erro de Escopo

Objetivo: Iniciei a implementação da syscall `getsyscallcount`, seguindo o fluxo de uma syscall existente.

Processo e Descobertas:

- Comecei mapeando os arquivos que precisaria alterar, com base na explicação do EP: `syscall.h`, `user.h`, `usys.S`, `proc.h`, `syscall.c` e `sysproc.c`.
- O primeiro passo foi definir um número para a nova syscall em `syscall.h` (`#define SYS_getsyscallcount 22`). Isso é essencial para que o kernel saiba qual função chamar.
- Em seguida, criei a "ponte" para o espaço de usuário em `user.h` e `usys.S`, para que os programas possam chamar a função.
- A maior mudança foi adicionar o array `int syscall_counts[MAXSYSCALLS];` na `struct proc` em `proc.h`. Decidi colocar aqui porque a contagem é por processo, então cada `struct proc` precisa ter seu próprio array.
- Comecei a escrever a função `sys_getsyscallcount()` em `sysproc.c`. Minha primeira ideia para validar o argumento foi verificar se o número da syscall estava dentro dos limites do array `syscalls`, que fica em `syscall.c`.

Sucessos e Reveses:

- **Sucesso:** A maioria das modificações iniciais foram simples e diretas.
- **Revés:** Ao tentar compilar, recebi o erro: `identifier "syscalls" is undefined` dentro de `sysproc.c`.
- **Solução/Aprendizado:** Percebi que o erro acontecia porque o array `syscalls` é declarado como `static` em `syscall.c`, o que significa que ele só é visível dentro daquele arquivo. Eu não podia acessá-lo de `sysproc.c`. A solução foi mudar a lógica de validação: em vez de usar o array, validei o número usando a constante `SYS_getsyscallcount`, que é o maior número de syscall e é visível globalmente por causa do `#include "syscall.h"`.

Entrada 2: A Avalanche de Erros de "Redefinition"

Objetivo: Fazer o código compilar após os primeiros erros e continuar a implementação.

Processo e Descobertas:

- Para usar a constante `MAXSYSCALLS` em `proc.h`, adicionei a linha `#include "syscall.h"`. Parecia a solução lógica para o erro de "identificador não definido".
- Ao compilar novamente, fui surpreendido por uma tela cheia de erros de `redefinition of 'struct...'`. O compilador reclamou de `struct segdesc`, `struct spinlock`, `struct trapframe`, etc.

Sucessos e Reveses:

- **Revés Gigante:** Minha "solução" criou um problema muito maior. Fiquei um bom tempo sem entender o que estava acontecendo.
- **Solução/Aprendizado:** Depois de pesquisar, entendi o conceito de **inclusão múltipla**. Ao incluir `syscall.h` em `proc.h`, eu criei uma cadeia onde outros arquivos (`console.c`, `bio.c`, etc.) acabavam incluindo os mesmos headers (`mmu.h`, `spinlock.h`) mais de uma vez. O compilador via a mesma `struct` sendo definida duas ou mais vezes e gerava o erro. O aprendizado aqui foi fundamental: a solução padrão em C para isso é usar **"include guards"**. Tive que editar arquivos como `mmu.h`, `spinlock.h` e `x86.h` para adicionar as diretivas `#ifndef`, `#define` e `#endif`. Isso garante que o conteúdo de um header seja incluído apenas uma vez, não importa quantas vezes seja chamado. Depois de fazer isso e rodar `make clean`, o código finalmente compilou!

Entrada 3: O Bug da Contagem Herdada no `fork()`

Objetivo: Executar o programa de teste e validar a lógica da syscall, especialmente o comportamento com `fork()`.

Processo e Descobertas:

- Criei meu programa de teste, `testesyscall.c`, e o adicionei ao `Makefile`.
- O programa rodou sem travar, o que foi um grande alívio. A contagem no processo pai parecia correta.
- No entanto, a saída do processo filho estava estranha. Enquanto as contagens de `fork()` e `getpid()` eram `0` (como esperado), a contagem de `write()` era um número alto, como `27`.

Sucessos e Reveses:

- **Sucesso:** A syscall funciona! O kernel não trava e retorna valores.

- **Revés:** A lógica estava errada para o processo filho. Ele não estava começando com os contadores zerados, como pedia o enunciado.
- **Solução/Aprendizado:** Este foi o "clique" final. Entendi que a função `fork()` cria o filho usando `copyproc`, que faz uma cópia exata da `struct proc` do pai. Isso significa que o filho estava **herdando** os valores dos contadores do pai no momento da criação. A solução foi ir até a função `fork()` em `proc.c` e, logo após a chamada a `copyproc`, adicionar um loop `for` para zerar explicitamente todos os valores no array `syscall_counts` do novo processo (o `np`). Após essa correção e uma nova compilação, a saída do teste ficou perfeita.